Notación Asintótica

Dra. Irene Olaya Ayaquica Martínez
Facultad de Ciencias de la Computación, BUAP
irene.ayaquica@correo.buap.mx

Cotas de complejidad – Medidas asintóticas

- Una vez vista la forma de calcular el tiempo de ejecución T de un algoritmo, nuestro propósito es intentar clasificar dichas funciones de forma que podamos compararlas.

- Para ello, vamos a definir clases de equivalencia, correspondientes a las funciones que "crecen de la misma forma".

- En las siguientes definiciones $\mathbb{N}$ denotará el conjunto de los números naturales y $\mathbb{R}$ el de los reales.

Cota Superior, Notación O

- Dada una función f, queremos estudiar aquellas funciones g que a lo sumo crecen tan de prisa como f.

- Al conjunto de tales funciones se le llama **cota superior de f** y lo denominamos $O(f)$.

- Conociendo la cota superior de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden superior al de la cota.

Cota Superior, Notación O

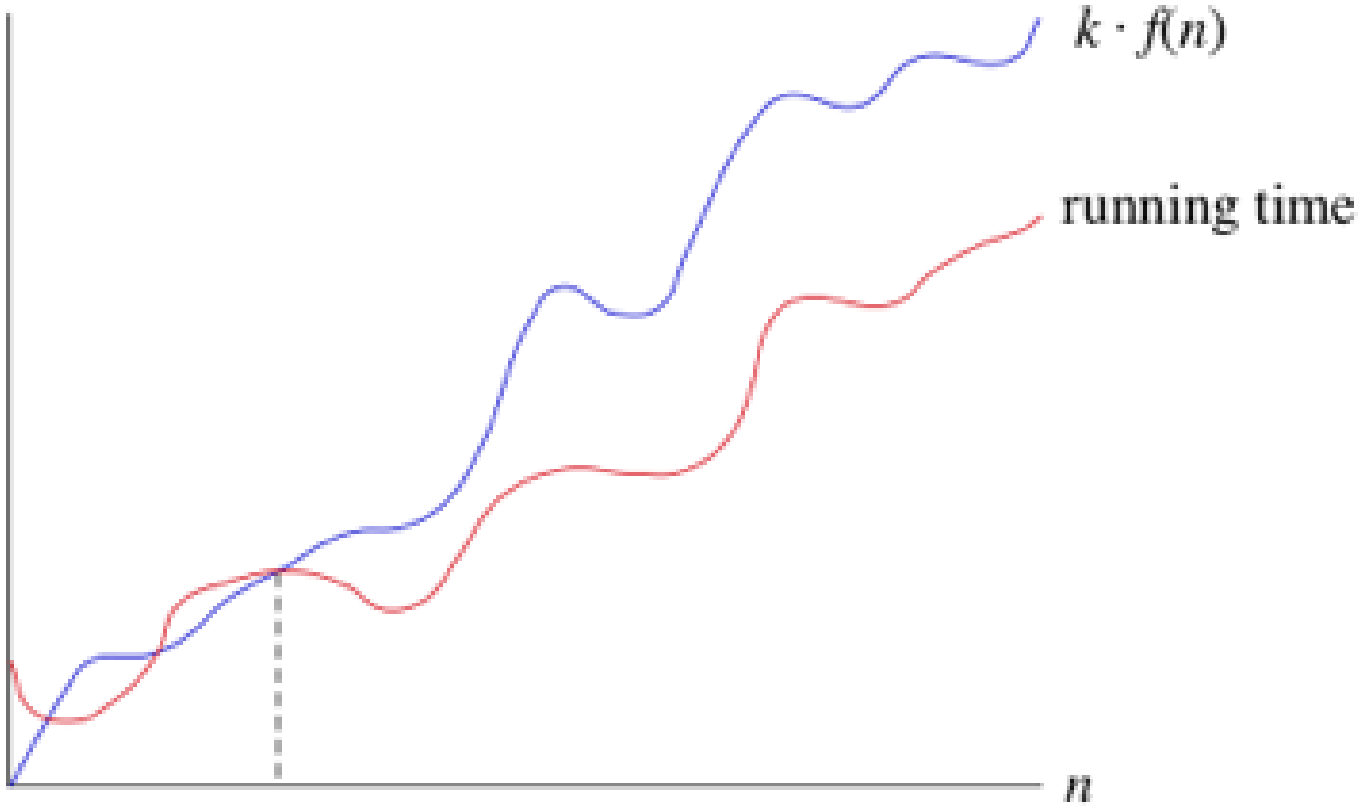
- **Def.** Sea $f: \mathbb{N} \to [0, \infty)$. Se define el conjunto de funciones de orden O de f como:

$$\mathrm{O}(f) = \{g: \mathbb{N} \to [0, \infty) | \exists c \in \mathbb{R}, c > 0, \exists n_0 \in \mathbb{N}, g(n) \leq cf(n), \forall n \geq n_0\}$$

- diremos que una función $t: \mathbb{N} \to [0, \infty)$ es de orden O de f si $t \in \mathrm{O}(f)$.

- $t \in \mathrm{O}(f)$ indica que t está acotada superiormente por algún múltiplo de f. Normalmente, estaremos interesados en la menor función f tal que $t \in \mathrm{O}(f)$.

Cota Superior, Notación O



Propiedades de O

1. Para cualquier función f se tiene que $f \in O(f)$.
2. $f \in O(g) \Rightarrow O(f) \subset O(g)$.
3. $O(f) = O(g) \Leftrightarrow f \in O(g) \wedge g \in O(f)$.
4. Si $f \in O(g) \wedge g \in O(h) \Rightarrow f \in O(h)$.

Propiedades de O

5. Si $f \in O(g) \wedge f \in O(h) \Rightarrow f \in O(\min(g,h))$.

6. Regla de la suma: Si $f_1 \in O(g) \wedge f_2 \in O(h) \Rightarrow f_1 + f_2 \in O(max(g,h))$.

7. Regla del producto: Si $f_1 \in O(g) \wedge f_2 \in O(h) \Rightarrow f_1 \cdot f_2 \in O(g \cdot h)$.

Propiedades de O

8. Si existe $\lim_{n\to\infty} \frac{f(n)}{g(n)} = k$, dependiendo de los valores que tome k obtenemos:
   a) Si $k \neq 0$ y $k < \infty$ entonces $O(f) = O(g)$.
   b) Si $k = 0$ entonces $f \in O(g)$, es decir, $O(f) \subset O(g)$ pero $g \notin O(f)$.
   c) Si $k = \infty$ entonces $f \notin O(g)$ pero $g \in O(f)$.

- Siempre escogeremos al representante más sencillo para cada clase; así, los órdenes de complejidad constante serán expresados por $O(1)$, los lineales por $O(n)$, etc.

Cota Inferior, Notación Ω

- Dada una función f, queremos estudiar aquellas funciones g que a lo sumo crecen tan lentamente como f.

- Al conjunto de tales funciones se le llama **cota inferior de f** y lo denominamos $\Omega(f)$.

- Conociendo la cota inferior de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden inferior al de la cota.

Cota Inferior, Notación Ω

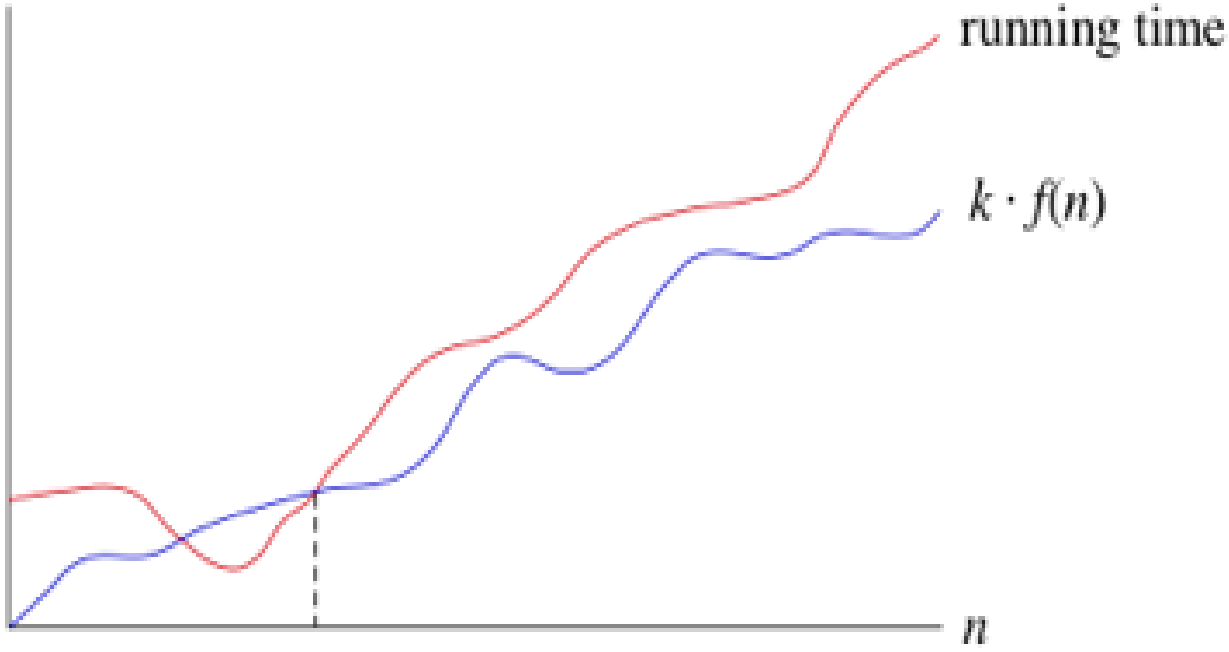
- **Def.** Sea $f: \mathbb{N} \to [0, \infty)$. Se define el conjunto de funciones de orden Ω de f como:

$$\Omega(f) = \{g: \mathbb{N} \to [0, \infty) | \exists c \in \mathbb{R}, c > 0, \exists n_0 \in \mathbb{N}, g(n) \geq cf(n), \forall n \geq n_0\}$$

- diremos que una función $t: \mathbb{N} \to [0, \infty)$ es de orden Ω de f si $t \in \Omega(f)$.

- $t \in \Omega(f)$ indica que t está acotada inferiormente por algún múltiplo de f. Normalmente, estaremos interesados en la mayor función f tal que $t \in \Omega(f)$.

Cota Inferior, Notación Ω



Propiedades de Ω

1. Para cualquier función f se tiene que $f \in \Omega(f)$.
2. $f \in \Omega(g) \Rightarrow \Omega(f) \subset \Omega(g)$.
3. $\Omega(f) = \Omega(g) \Leftrightarrow f \in \Omega(g) \wedge g \in \Omega(f)$.
4. Si $f \in \Omega(g) \wedge g \in \Omega(h) \Rightarrow f \in \Omega(h)$.

Propiedades de Ω

5. Si $f \in \Omega(g) \wedge f \in \Omega(h) \Rightarrow f \in \Omega(\max(g,h))$.

6. Regla de la suma: Si $f_1 \in \Omega(g) \wedge f_2 \in \Omega(h) \Rightarrow f_1 + f_2 \in \Omega(g+h)$.

7. Regla del producto: Si $f_1 \in \Omega(g) \wedge f_2 \in \Omega(h) \Rightarrow f_1 \cdot f_2 \in \Omega(g \cdot h)$.

Propiedades de Ω

8. Si existe $\lim_{n\to\infty} \frac{f(n)}{g(n)} = k$, dependiendo de los valores que tome k obtenemos:
   a) Si $k \neq 0$ y $k < \infty$ entonces $\Omega(f) = \Omega(g)$.
   b) Si $k = 0$ entonces $g \in \Omega(f)$, es decir, $\Omega(g) \subset \Omega(f)$ pero $f \notin \Omega(g)$.
   c) Si $k = \infty$ entonces $g \notin \Omega(f)$ pero $f \in \Omega(g)$.

- Siempre escogeremos al representante más sencillo para cada clase; así, los órdenes de complejidad constante serán expresados por $\Omega(1)$, los lineales por $\Omega(n)$, etc.

Orden Exacto, Notación Θ

- Deseamos encontrar el conjunto de funciones que crecen asintóticamente de la misma forma.

- **Def.** Sea $f: \mathbb{N} \to [0, \infty)$. Se define el conjunto de funciones de orden Θ de f como:

$$\Theta(f) = O(f) \cap \Omega(f)$$

o lo que es igual

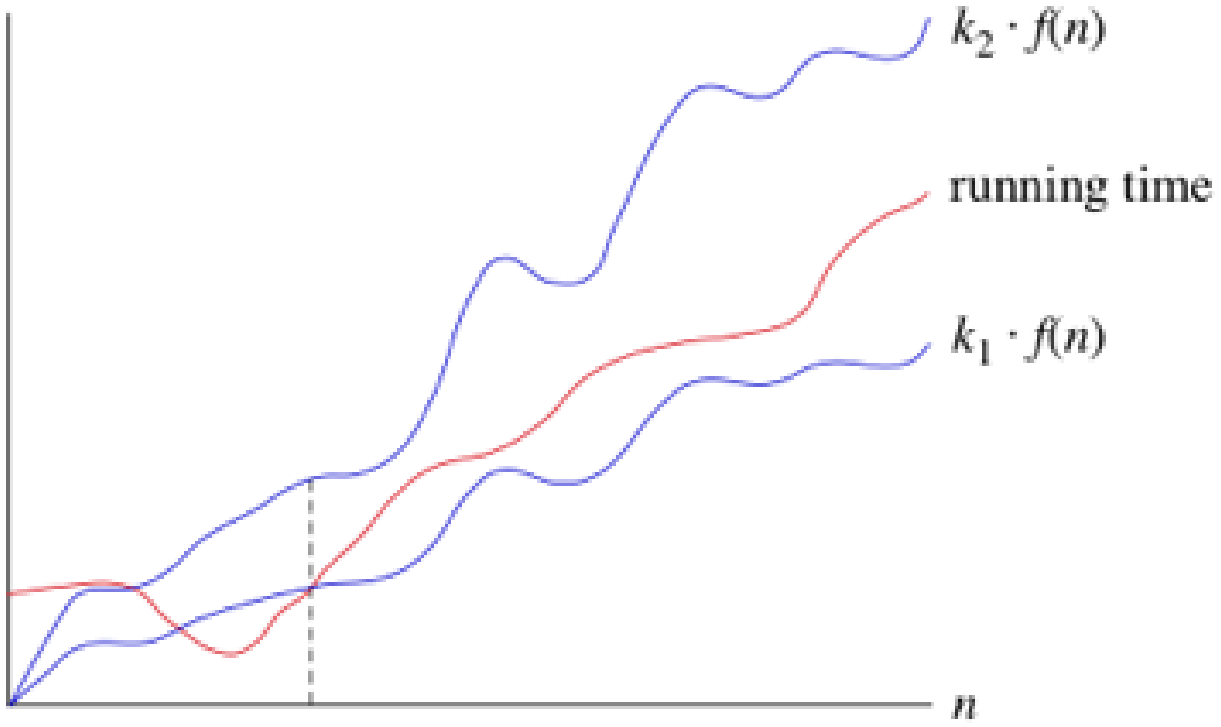
$$\Theta(f) = \{g: \mathbb{N} \to [0, \infty) | \exists c, d \in \mathbb{R}, c, d > 0, \exists n_0 \in \mathbb{N}, cf(n) \leq g(n) \leq df(n), \forall n \geq n_0\}$$

Orden Exacto, Notación Θ

- diremos que una función $t: \mathbb{N} \to [0, \infty)$ es de orden Θ de f si $t \in \Theta(f)$.

- $t \in \Theta(f)$ indica que t está acotada tanto superior como inferiormente por múltiplos de f, es decir, que t y f crecen de la misma forma.

Orden Exacto, Notación Θ



Propiedades de Θ

1. Para cualquier función f se tiene que $f \in \Theta(f)$.
2. $f \in \Theta(g) \Rightarrow \Theta(f) \subset \Theta(g)$.
3. $\Theta(f) = \Theta(g) \Leftrightarrow f \in \Theta(g) \wedge g \in \Theta(f)$.
4. Si $f \in \Theta(g) \wedge g \in \Theta(h) \Rightarrow f \in \Theta(h)$.

Propiedades de Θ

5. Regla de la suma: Si $f_1 \in \Theta(g) \wedge f_2 \in \Theta(h) \Rightarrow f_1 + f_2 \in \Theta(\max(g, h))$.

6. Regla del producto: Si $f_1 \in \Theta(g) \wedge f_2 \in \Theta(h) \Rightarrow f_1 \cdot f_2 \in \Theta(g \cdot h)$.

Propiedades de Θ

7. Si existe $\lim_{n\to\infty} \frac{f(n)}{g(n)} = k$, dependiendo de los valores que tome k obtenemos:
   a) Si $k \neq 0$ y $k < \infty$ entonces $\Theta(f) = \Theta(g) \rightarrow f \in \Theta(g)$
   b) Si $k = 0$ entonces los órdenes exactos de f y g son distintos $\rightarrow f \in O(g)$ pero $f \notin \Theta(g)$.
   c) Si $k = \infty$ entonces $f \in \Omega(g)$ pero $f \notin \Theta(g)$.

Orden de complejidad de un algoritmo

- Dado un algoritmo diremos que su orden de complejidad es $\boldsymbol{O(f)}$ si su tiempo de ejecución para el ***peor caso*** es de orden O de f, es decir, $T_p(n)$ es de orden $\mathrm{O}(f)$.

- Su orden de complejidad es $\boldsymbol{\Omega(f)}$ si su tiempo de ejecución para el ***mejor caso*** es de orden Ω de g, es decir, $T_m(n)$ es de orden $\Omega(f)$.

- Y es de orden exacto $\boldsymbol{\Theta(f)}$ si su tiempo de ejecución en el ***caso medio*** $T_{1/2}(n)$ es de orden $\Theta(f)$.